Lecture 7: Time Complexity, P vs. NP

Alessandro Chiesa

EPFL School of Computer and Communication Sciences

Time Complexity

Computability: worst case

Number of configurations TM needs to reach an accept/reject state on this input

Recognizers for a language L;
Which one is better?

	M_A	M_B
ε	$< \infty$	$< \infty$
0	$< \infty$	$< \infty$
1	$< \infty$	∞
00	$< \infty$	$< \infty$
01	$< \infty$	$< \infty$
10	$< \infty$	$< \infty$
11	$< \infty$	$< \infty$
000	$< \infty$	∞
001	$< \infty$	$< \infty$
010	$< \infty$	$< \infty$
$\vdots$	$\vdots$	$\vdots$

Does not halt on this input and, hence, is not a decider

Computability: worst case

Number of configurations TM needs to reach an accept/reject state on this input

Recognizers for a language L;
Which one is better?

	M_A	M_B
ε	$< \infty$	$< \infty$
0	$< \infty$	$< \infty$
1	$< \infty$	∞
00	$< \infty$	$< \infty$
01	$< \infty$	$< \infty$
10	$< \infty$	$< \infty$
11	$< \infty$	$< \infty$
000	$< \infty$	∞
001	$< \infty$	$< \infty$
010	$< \infty$	$< \infty$
$\vdots$	$\vdots$	$\vdots$

Does not halt on this input and, hence, is not a decider

In the rest of the course material we just consider decidable languages

A decidable language L

Number of configurations TM needs to reach an accept/reject state on this input

Deciders for L:

Inputs	M_A	M_B	M_C	M_D	M_E	M_F	M_G	M_H	
ε	2	2	5	2	3	4	2	2	⋯
0	2	5	12	2	3	5	12	5	⋯
1	20	12	14	13	8	19	2	9	⋯
00	32	14	18	9	18	3	5	90	⋯
01	12	21	56	8	12	18	18	30	⋯
10	21	22	26	15	11	12	32	15	⋯
11	11	12	25	100	13	48	98	29	⋯
000	320	201	159	201	190	200	180	65	⋯
001	211	208	190	200	189	301	219	82	⋯
010	328	271	214	441	193	208	109	77	⋯
011	227	261	191	201	188	107	211	207	⋯
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋱

A decidable language L

Number of configurations TM needs to reach an accept/reject state on this input

Deciders for L:

Inputs	M_A	M_B	M_C	M_D	M_E	M_F	M_G	M_H	
ε	2	2	5	2	3	4	2	2	⋯
0	2	5	12	2	3	5	12	5	⋯
1	20	12	14	13	8	19	2	9	⋯
00	32	14	18	9	18	3	5	90	⋯
01	12	21	56	8	12	18	18	30	⋯
10	21	22	26	15	11	12	32	15	⋯
11	11	12	25	100	13	48	98	29	⋯
000	320	201	159	201	190	200	180	65	⋯
001	211	208	190	200	189	301	219	82	⋯
010	328	271	214	441	193	208	109	77	⋯
011	227	261	191	201	188	107	211	207	⋯
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋱

How to compare different deciders?

Two deciders for L

	M_A	
ε	2	2
0	4	12
1	12	
00	15	110
01	22	
10	17	
11	110	
000	49	300
001	34	
010	38	
011	300	
⋮		
	⋮	

	M_B	
ε	2	2
0	9	10
1	10	
00	40	45
01	30	
10	45	
11	39	
000	73	85
001	77	
010	85	
011	80	
⋮		
	⋮	

Worst case in this group

Running time of a TM

Definition: Let M be a TM that halts on all inputs (decider). The running time or time complexity of M is the function $t : \mathbb{N} \to \mathbb{N}$ where

$$t(n) = \max_{w \in \Sigma^* : |w| = n} \text{number of steps } M \text{ takes on } w$$

Running time of a TM

Definition: Let M be a TM that halts on all inputs (decider). The running time or time complexity of M is the function $t : \mathbb{N} \to \mathbb{N}$ where

$$t(n) = \max_{w \in \Sigma^* : |w| = n} \text{number of steps } M \text{ takes on } w$$

- M runs in time $t(n)$
- n represents the input length

$$t(0) = 2$$
$$t(1) = 10$$
$$t(2) = 45$$
$$t(3) = 85$$

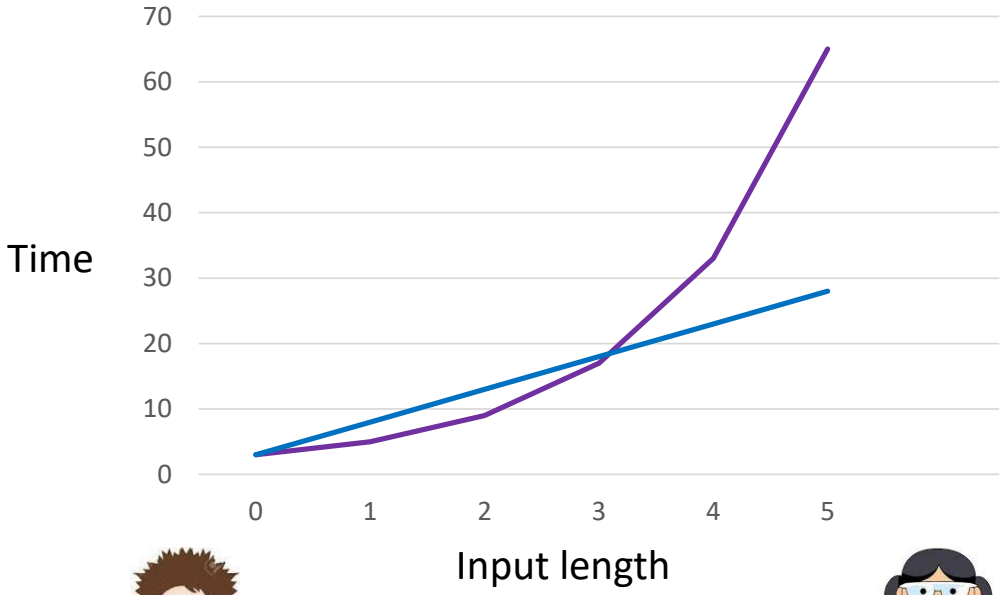
ε	2	2
0	9	10
1	10	
00	21	45
01	30	
10	45	
11	33	
000	73	85
001	77	
010	85	
011	80	
$\vdots$		

Two deciders for L

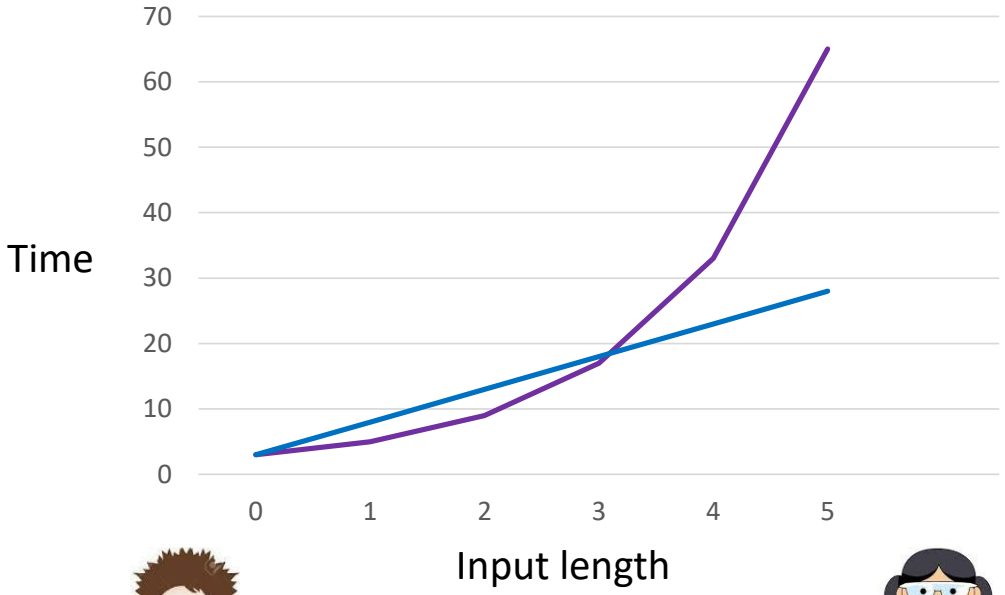
M_A	
Length 0	$t_1(0) = 3$
Length 1	$t_1(1) = 5$
Length 2	$t_1(2) = 9$
Length 3	$t_1(3) = 17$
Length 4	$t_1(4) = 33$
Length 5	$t_1(5) = 65$
$\vdots$	$\vdots$

M_B	
Length 0	$t_2(0) = 3$
Length 1	$t_2(1) = 8$
Length 2	$t_2(2) = 13$
Length 3	$t_2(3) = 18$
Length 4	$t_2(4) = 23$
Length 5	$t_2(5) = 28$
$\vdots$	$\vdots$

$t_1(n) = 2^{n+1} + 1$ vs $t_2(n) = 5n + 3$



$t_1(n) = 2^{n+1} + 1$ vs $t_2(n) = 5n + 3$



How to compare running time functions?

The Growth of Functions

- Three TMs: A, B, C with different running times in ms (say 1 step is 1 ms)

	A ($1000 \log_2 n$ ms)	B ($2n^2$ ms)	C (2^n ms)

The Growth of Functions

- Three TMs: A, B, C with different running times in ms (say 1 step is 1 ms)

	A ($1000 \log_2 n$ ms)	B ($2n^2$ ms)	C (2^n ms)
$n = 2$	1 s	8 ms	4 ms
$n = 4$	2 s	32 ms	16 ms
$n = 8$	3 s	128 ms	256 ms
$n = 16$	4 s	512 ms	1 m 5s 536 ms

The Growth of Functions

- Three TMs: A, B, C with different running times in ms (say 1 step is 1 ms)

	A ($1000 \log_2 n$ ms)	B ($2n^2$ ms)	C (2^n ms)
$n = 2$	1 s	8 ms	4 ms
$n = 4$	2 s	32 ms	16 ms
$n = 8$	3 s	128 ms	256 ms
$n = 16$	4 s	512 ms	1 m 5s 536 ms
$n = 32$	5 s	2 s 48 ms	$\approx$ 49 days 18h

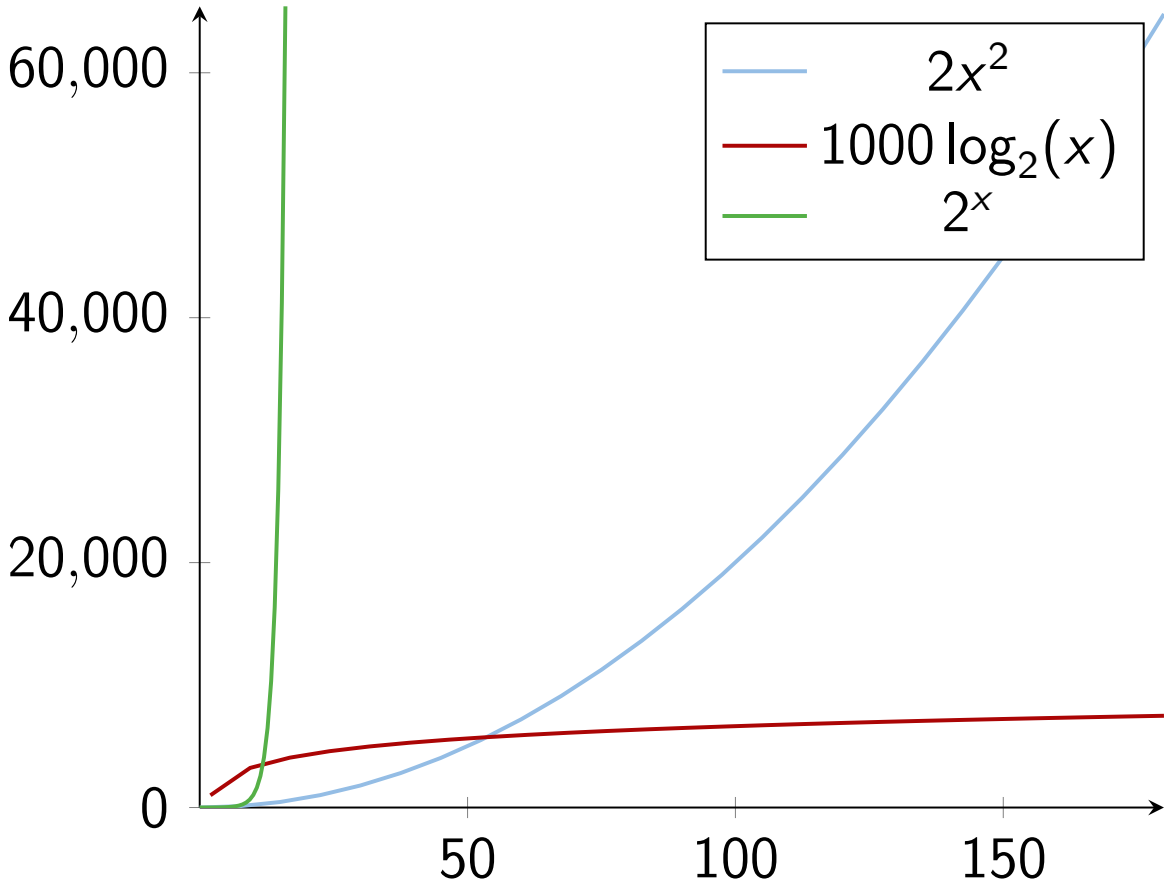
The Growth of Functions

- Three TMs: A, B, C with different running times in ms (say 1 step is 1 ms)

	A ($1000 \log_2 n$ ms)	B ($2n^2$ ms)	C (2^n ms)
$n = 2$	1 s	8 ms	4 ms
$n = 4$	2 s	32 ms	16 ms
$n = 8$	3 s	128 ms	256 ms
$n = 16$	4 s	512 ms	1 m 5s 536 ms
$n = 32$	5 s	2 s 48 ms	$\approx$ 49 days 18h
$n = 64$	6 s	8 s 192 ms	$>$ age of the universe

The Growth of Functions

- Three TMs: A, B, C with different running times in ms (say 1 step is 1 ms)



Big-O and Small-o notation

Big-O and Small-o notation

Definition (Big-O): Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We say $f(n) = O(g(n))$ if

$$\exists C > 0, n_0 \in \mathbb{N} \text{ s.t. } \qquad \forall n \geq n_0 \;\; f(n) \leq C \cdot g(n)$$

Big-O and Small-o notation

Definition (Big-O): Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We say $f(n) = O(g(n))$ if

$$\exists C > 0, n_0 \in \mathbb{N} \text{ s.t. } \qquad \forall n \geq n_0 \quad f(n) \leq C \cdot g(n)$$

Examples:

- $5n^3 + 1 = O(2^n)$? YES
- $5n^3 + 1 = O(20n + 5)$? NO

Big-O and Small-o notation

Definition (Big-O): Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We say $f(n) = O(g(n))$ if

$$\exists C > 0, n_0 \in \mathbb{N} \text{ s.t.} \qquad \forall n \geq n_0 \;\; f(n) \leq C \cdot g(n)$$

Examples:

- $5n^3 + 1 = O(2^n)$? YES
- $5n^3 + 1 = O(20n + 5)$? NO

Definition (Small-o): Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We say $f(n) = o(g(n))$ if

$$\forall c > 0, \exists n_0 \in \mathbb{N} \text{ s.t.} \qquad \forall n \geq n_0 \;\; f(n) < c \cdot g(n)$$

Big-O and Small-o notation

Definition (Big-O): Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We say $f(n) = O(g(n))$ if

$$\exists C > 0, n_0 \in \mathbb{N} \text{ s.t. } \qquad \forall n \geq n_0 \quad f(n) \leq C \cdot g(n)$$

Examples:

- $5n^3 + 1 = O(2^n)$? YES
- $5n^3 + 1 = O(20n + 5)$? NO

Definition (Small-o): Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We say $f(n) = o(g(n))$ if

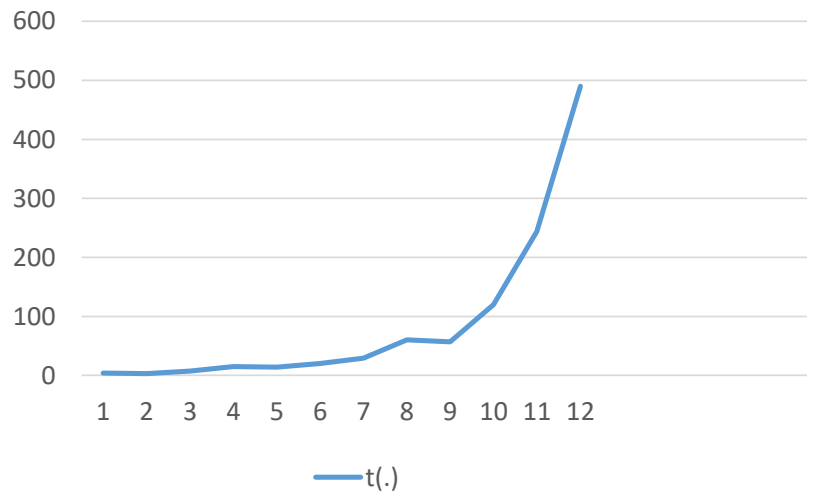
$$\forall c > 0, \exists n_0 \in \mathbb{N} \text{ s.t. } \qquad \forall n \geq n_0 \quad f(n) < c \cdot g(n)$$

Examples:

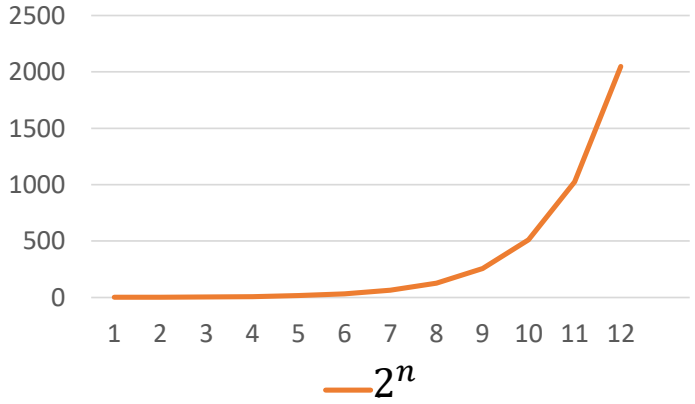
- $\sqrt{n} = o(n)$? YES
- $f(n) = o(f(n))$? NO

To summarize. . .

M		
2	Length 0	$t(0) = 2$
4 3	Length 1	$t(1) = 4$
2 3 ⋮	Length 2	$t(2) = 3$
4 ⋮ 7 ⋮	Length 3	$t(3) = 7$
6 10 ⋮ 15 ⋮	Length 4	$t(4) = 15$
10 8 ⋮ 14 ⋮	Length 5	$t(5) = 14$ ⋮
	⋮	



$t(n) = O(2^n)$



Time Complexity

Definition: Time complexity class

$TIME(t(n)) = \{L \subseteq \Sigma^* \mid L \text{ is decided by a TM with running time } O(t(n))\}$

Time Complexity

Definition: Time complexity class

$TIME(t(n)) = \{L \subseteq \Sigma^* \mid L \text{ is decided by a TM with running time } O(t(n))\}$

Example:

$TIME(n) \subseteq TIME(n^2) \subseteq \cdots \subseteq TIME(2^{\sqrt{n}}) \subseteq TIME(2^n) \subseteq TIME(2^{2^n}) \ldots$

Time Complexity

Definition: Time complexity class

$TIME(t(n)) = \{L \subseteq \Sigma^* \mid L \text{ is decided by a TM with running time } O(t(n))\}$

Example:
$TIME(n) \subseteq TIME(n^2) \subseteq \cdots \subseteq TIME(2^{\sqrt{n}}) \subseteq TIME(2^n) \subseteq TIME(2^{2^n}) \ldots$

Example: $REGULAR \subseteq TIME(n)$

The complexity class **P** and efficiency

Definition: **P** is the class of languages that are decidable in polynomial time on a (deterministic) Turing machine. In other words,

$$\mathbf{P} = \bigcup_{k=1}^{\infty} TIME(n^k)\,.$$

The complexity class **P** and efficiency

Definition: **P** is the class of languages that are decidable in polynomial time on a (deterministic) Turing machine. In other words,

$$\mathbf{P} = \bigcup_{k=1}^{\infty} TIME(n^k)\,.$$

Some languages in P:

- $\{\langle A \rangle \ : \ A \text{ is a sorted array of integers}\}$
- $\{\langle G, s, t \rangle \ : \ s \text{ and } t \text{ are vertices connected in graph } G\}$ (Breadth-First Search)
- $\{\langle G \rangle \ : \ G \text{ is a connected graph}\}$

The complexity class **P** and efficiency

Definition: **P** is the class of languages that are decidable in polynomial time on a (deterministic) Turing machine. In other words,

$$\mathbf{P} = \bigcup_{k=1}^{\infty} TIME(n^k)\,.$$

Robust definition: **P** is invariant for all models of computation that are polynomially equivalent to deterministic TM

To show $L \in \mathbf{P}$ enough to describe deterministic algorithm in words and in your favorite programming/pseudo-code language.

The complexity class **P** and efficiency

Definition: **P** is the class of languages that are decidable in polynomial time on a (deterministic) Turing machine. In other words,

$$\mathbf{P} = \bigcup_{k=1}^{\infty} TIME(n^k)\,.$$

Robust definition: **P** is invariant for all models of computation that are polynomially equivalent to deterministic TM

To show $L \in \mathbf{P}$ enough to describe deterministic algorithm in words and in your favorite programming/pseudo-code language.

Extended Church–Turing thesis: **P** corresponds to the class of problems that are "realistically" solvable in our universe

Possible challenges: randomisation and quantum?

NP: Verification vs. Search

SAT-verify and SAT

Conjunctive Normal Form (CNF) Formula:

$$\varphi_1 = (\overline{x} \vee \overline{y} \vee z_0) \wedge (x \vee \overline{y} \vee z_1) \wedge (\overline{x} \vee y \vee z_2) \wedge (x \vee y \vee z_3)$$

$$\varphi_2 = \overline{x_1} \wedge (x_1 \vee \overline{x_2}) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee x_2 \vee x_3 \vee \overline{x_4})$$

$$\varphi_3 = \overline{x_1} \wedge (x_1 \vee \overline{x_2}) \wedge (x_1 \vee x_2)$$

SAT-verify and SAT

Conjunctive Normal Form (CNF) Formula:

$$\varphi_1 = (\overline{x} \vee \overline{y} \vee z_0) \wedge (x \vee \overline{y} \vee z_1) \wedge (\overline{x} \vee y \vee z_2) \wedge (x \vee y \vee z_3)$$

$$\varphi_2 = \overline{x_1} \wedge (x_1 \vee \overline{x_2}) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee x_2 \vee x_3 \vee \overline{x_4})$$

$$\varphi_3 = \overline{x_1} \wedge (x_1 \vee \overline{x_2}) \wedge (x_1 \vee x_2)$$

- CNF Formula: AND of Clauses
- Clause: OR of Literals
- Literal: variable or its negation

SAT-verify and SAT

Conjunctive Normal Form (CNF) Formula:

$$\varphi_1 = (\overline{x} \vee \overline{y} \vee z_0) \wedge (x \vee \overline{y} \vee z_1) \wedge (\overline{x} \vee y \vee z_2) \wedge (x \vee y \vee z_3)$$

$$\varphi_2 = \overline{x_1} \wedge (x_1 \vee \overline{x_2}) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee x_2 \vee x_3 \vee \overline{x_4})$$

$$\varphi_3 = \overline{x_1} \wedge (x_1 \vee \overline{x_2}) \wedge (x_1 \vee x_2)$$

- CNF Formula: AND of Clauses
- Clause: OR of Literals
- Literal: variable or its negation

Satisfying assignment: Boolean assignment to variables which makes the formula TRUE.

Check: φ_1 has 32 satisfying assignments, φ_2 has only one, φ_3 has zero.

SAT-verify and SAT

Conjunctive Normal Form (CNF) Formula:

$$\varphi_1 = (\overline{x} \vee \overline{y} \vee z_0) \wedge (x \vee \overline{y} \vee z_1) \wedge (\overline{x} \vee y \vee z_2) \wedge (x \vee y \vee z_3)$$

$$\varphi_2 = \overline{x_1} \wedge (x_1 \vee \overline{x_2}) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee x_2 \vee x_3 \vee \overline{x_4})$$

$$\varphi_3 = \overline{x_1} \wedge (x_1 \vee \overline{x_2}) \wedge (x_1 \vee x_2)$$

- CNF Formula: AND of Clauses
- Clause: OR of Literals
- Literal: variable or its negation

Satisfying assignment: Boolean assignment to variables which makes the formula TRUE.

Check: φ_1 has 32 satisfying assignments, φ_2 has only one, φ_3 has zero.

A formula is **satisfiable** if it has at least one satisfying assignment.

SAT-verify and SAT

$$\text{SAT-verify} = \{\langle \varphi, C \rangle \ : \ C \text{ is a satisfying assignment of } \varphi\}$$

SAT-verify and SAT

$$\text{SAT-verify} = \{\langle \varphi, C \rangle \ : \ C \text{ is a satisfying assignment of } \varphi\}$$

Is SAT-verify in P?

SAT-verify and SAT

$$\text{SAT-verify} = \{\langle \varphi, C \rangle \ : \ C \text{ is a satisfying assignment of } \varphi\}$$

Is SAT-verify in P? Yes!

1. Substitute for literals according to C.
2. Check that every clause has at least one TRUE literal.

SAT-verify and SAT

$$\text{SAT-verify} = \{\langle \varphi, C \rangle \ : \ C \text{ is a satisfying assignment of } \varphi\}$$

Is SAT-verify in P? Yes!

1. Substitute for literals according to C.
2. Check that every clause has at least one TRUE literal.

$$\begin{aligned} \text{SAT} &= \{\langle \varphi \rangle \ : \ \varphi \text{ is satisfiable}\} \\ &= \{\langle \varphi \rangle \ : \ \exists C \text{ such that } \langle \varphi, C \rangle \in \text{SAT-verify}\} \end{aligned}$$

SAT-verify and SAT

$$\text{SAT-verify} = \{\langle \varphi, C \rangle \ : \ C \text{ is a satisfying assignment of } \varphi\}$$

Is SAT-verify in P? Yes!

1. Substitute for literals according to C.
2. Check that every clause has at least one TRUE literal.

$$\begin{aligned} \text{SAT} &= \{\langle \varphi \rangle \ : \ \varphi \text{ is satisfiable}\} \\ &= \{\langle \varphi \rangle \ : \ \exists C \text{ such that } \langle \varphi, C \rangle \in \text{SAT-verify}\} \end{aligned}$$

Is SAT in P?

SAT-verify and SAT

$$\text{SAT-verify} = \{\langle \varphi, C \rangle \ : \ C \text{ is a satisfying assignment of } \varphi\}$$

Is SAT-verify in P? Yes!

1. Substitute for literals according to C.
2. Check that every clause has at least one TRUE literal.

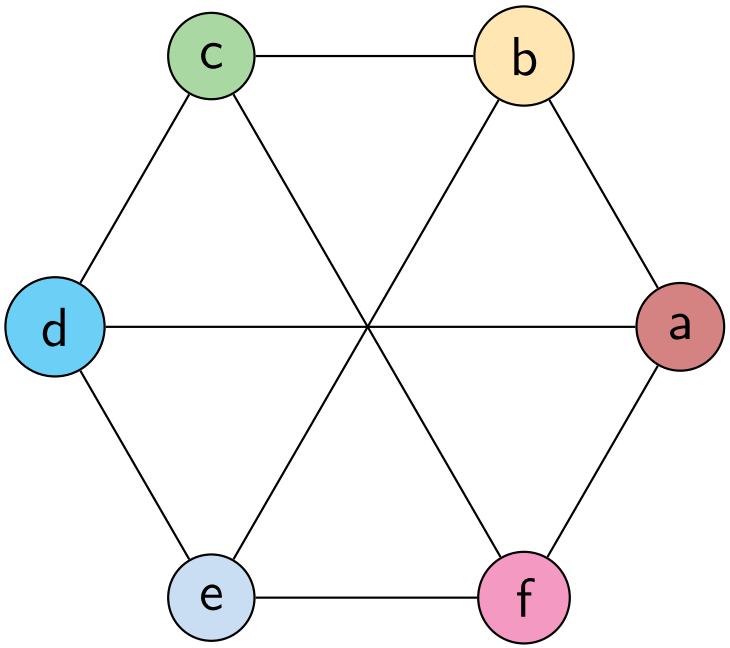
$$\begin{aligned} \text{SAT} &= \{\langle \varphi \rangle \ : \ \varphi \text{ is satisfiable}\} \\ &= \{\langle \varphi \rangle \ : \ \exists C \text{ such that } \langle \varphi, C \rangle \in \text{SAT-verify}\} \end{aligned}$$

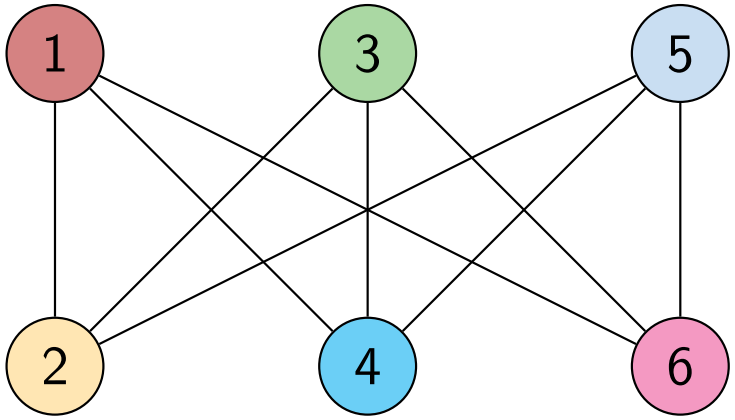
Is SAT in P?

Decider for SAT:

1. For each assignment C:
   - If $\langle \varphi, C \rangle \in$ SAT-verify, ACCEPT φ.
2. REJECT φ.

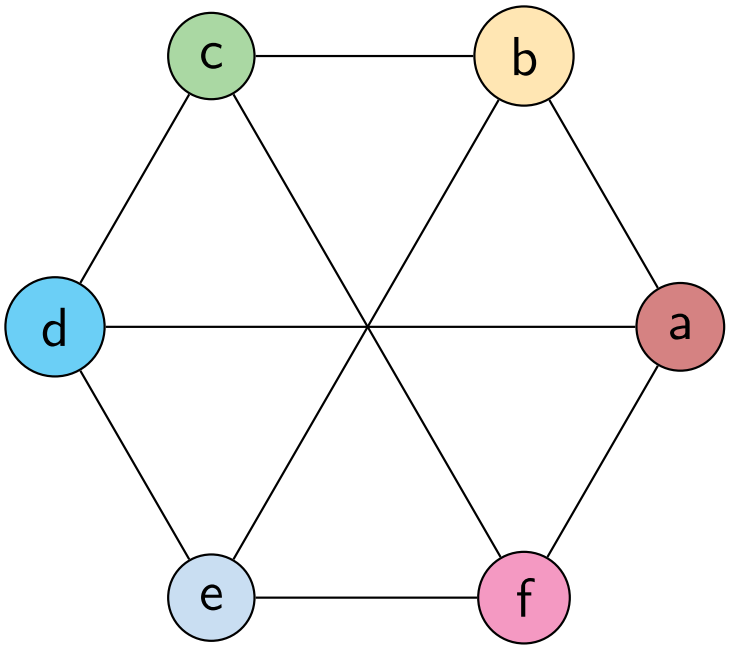
GI-verify and GI

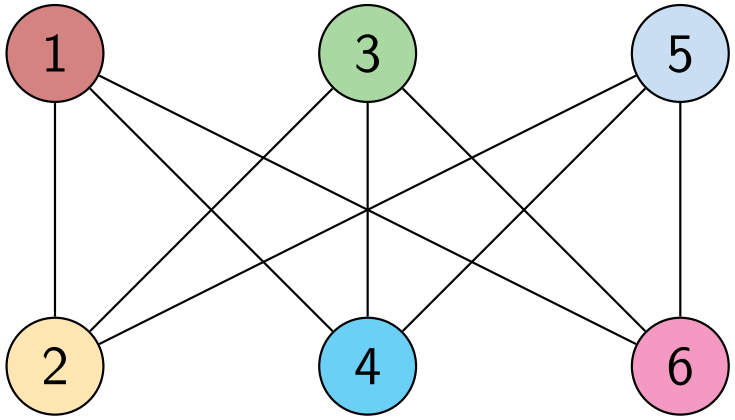




Graph Isomorphism: Bijection $f : V(G_1) \longrightarrow V(G_2)$ which preserves adjacency: $\{u, v\} \in E(G_1) \Leftrightarrow \{f(u), f(v)\} \in E(G_2)$

GI-verify and GI

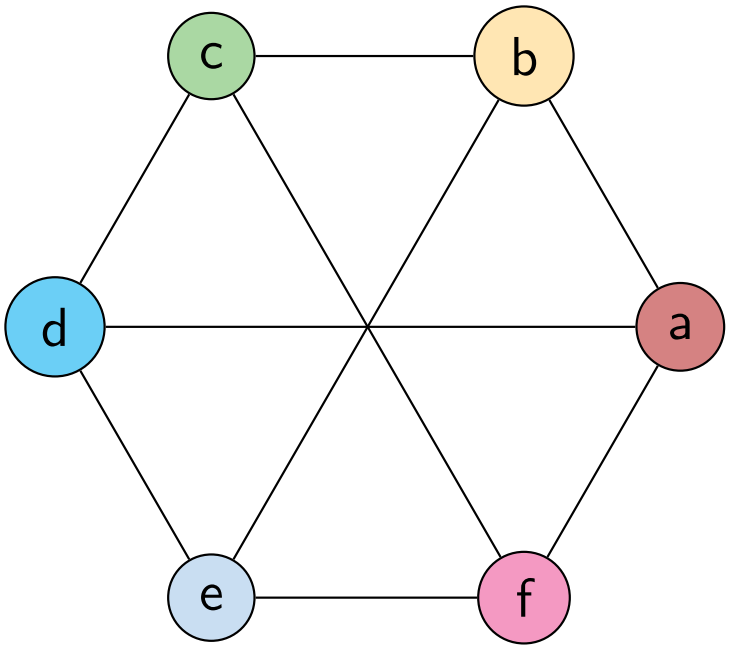


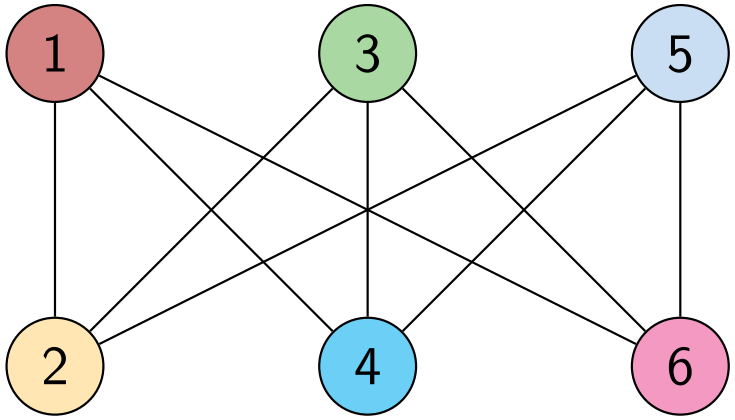


Graph Isomorphism: Bijection $f : V(G_1) \longrightarrow V(G_2)$ which preserves adjacency: $\{u, v\} \in E(G_1) \Leftrightarrow \{f(u), f(v)\} \in E(G_2)$

Eg. $a \to 1$ $b \to 2$ $c \to 3$ $d \to 4$ $e \to 5$ $f \to 6$ in the graphs above.

GI-verify and GI





Graph Isomorphism: Bijection $f : V(G_1) \longrightarrow V(G_2)$ which preserves adjacency: $\{u, v\} \in E(G_1) \Leftrightarrow \{f(u), f(v)\} \in E(G_2)$

Eg. $a \to 1$ $b \to 2$ $c \to 3$ $d \to 4$ $e \to 5$ $f \to 6$ in the graphs above.

Two graphs are **isomorphic** if they have at least one graph isomorphism.

GI-verify and GI

$$\text{GI-verify} = \{\langle G_1, G_2, C\rangle \ : \ C : V(G_1) \longrightarrow V(G_2) \text{ is a graph isomorphism}\}$$

GI-verify and GI

$$\text{GI-verify} = \{\langle G_1, G_2, C\rangle \ : \ C : V(G_1) \longrightarrow V(G_2) \text{ is a graph isomorphism}\}$$

Is GI-verify in P?

GI-verify and GI

$$\text{GI-verify} = \{\langle G_1, G_2, C\rangle \ : \ C : V(G_1) \longrightarrow V(G_2) \text{ is a graph isomorphism}\}$$

Is GI-verify in P? Yes!

1. Check that C is a bijection: For each $u, v \in V(G_1)$:
   - Check $\{u, v\} \in E(G_1) \Leftrightarrow \{C(u), C(v)\} \in E(G_2)$.

GI-verify and GI

$$\text{GI-verify} = \{\langle G_1, G_2, C\rangle \ : \ C : V(G_1) \longrightarrow V(G_2) \text{ is a graph isomorphism}\}$$

Is GI-verify in P? Yes!

1. Check that C is a bijection: For each $u, v \in V(G_1)$:
   - Check $\{u, v\} \in E(G_1) \Leftrightarrow \{C(u), C(v)\} \in E(G_2)$.

$$\begin{aligned} \text{GI} &= \{\langle G_1, G_2\rangle \ : \ G_1 \text{ and } G_2 \text{ are isomorphic}\} \\ &= \{\langle G_1, G_2\rangle \ : \ \exists C \text{ such that } \langle G_1, G_2, C\rangle \in \text{GI-verify}\} \end{aligned}$$

GI-verify and GI

$$\text{GI-verify} = \{\langle G_1, G_2, C\rangle \ : \ C : V(G_1) \longrightarrow V(G_2) \text{ is a graph isomorphism}\}$$

Is GI-verify in P? Yes!

1. Check that C is a bijection: For each $u, v \in V(G_1)$:
   - Check $\{u, v\} \in E(G_1) \Leftrightarrow \{C(u), C(v)\} \in E(G_2)$.

$$\begin{aligned} \text{GI} &= \{\langle G_1, G_2\rangle \ : \ G_1 \text{ and } G_2 \text{ are isomorphic}\} \\ &= \{\langle G_1, G_2\rangle \ : \ \exists C \text{ such that } \langle G_1, G_2, C\rangle \in \text{GI-verify}\} \end{aligned}$$

Is GI in P?

GI-verify and GI

$$\text{GI-verify} = \{\langle G_1, G_2, C\rangle \ : \ C : V(G_1) \longrightarrow V(G_2) \text{ is a graph isomorphism}\}$$

Is GI-verify in P? Yes!

1. Check that C is a bijection: For each $u, v \in V(G_1)$:
   - Check $\{u, v\} \in E(G_1) \Leftrightarrow \{C(u), C(v)\} \in E(G_2)$.

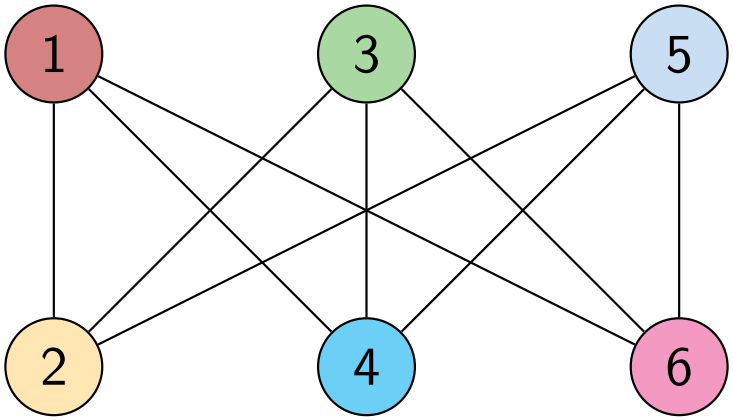
$$\begin{aligned} \text{GI} &= \{\langle G_1, G_2\rangle \ : \ G_1 \text{ and } G_2 \text{ are isomorphic}\} \\ &= \{\langle G_1, G_2\rangle \ : \ \exists C \text{ such that } \langle G_1, G_2, C\rangle \in \text{GI-verify}\} \end{aligned}$$

Is GI in P?

Decider for GI:

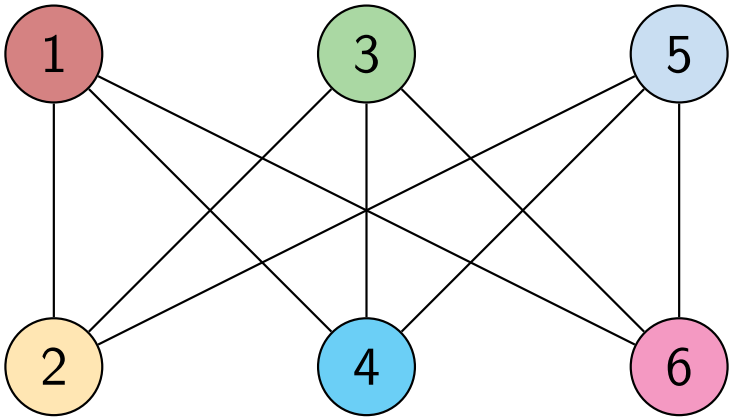
1. For each function $C : V(G_1) \longrightarrow V(G_2)$:
   - If $\langle G_1, G_2, C\rangle \in$ GI-verify, ACCEPT $\langle G_1, G_2\rangle$.
2. REJECT $\langle G_1, G_2\rangle$.

INDSET-verify and INDSET



Independent Set: Subset $S \subseteq V(G)$ such that no two vertices in S are adjacent in G.

INDSET-verify and INDSET



Independent Set: Subset $S \subseteq V(G)$ such that no two vertices in S are adjacent in G.

Eg. $\{1, 3, 5\}$, $\{2, 4\}$, $\{6\}$, $\emptyset$, etc. in the graph above.

INDSET-verify and INDSET

$$\text{INDSET-verify} = \{\langle G, k, C\rangle \ : \ C \text{ is an independent set of size } k \text{ in } G\}$$

INDSET-verify and INDSET

$$\text{INDSET-verify} = \{\langle G, k, C\rangle \ : \ C \text{ is an independent set of size } k \text{ in } G\}$$

Is INDSET-verify in P?

INDSET-verify and INDSET

INDSET-verify $= \{\langle G, k, C\rangle \ : \ C$ is an independent set of size k in $G\}$

Is INDSET-verify in P? Yes!

1. Check that $|C| = k$.
2. For each $u, v \in C$:
   - Check $\{u, v\} \notin E(G)$.

INDSET-verify and INDSET

$$\text{INDSET-verify} = \{\langle G, k, C\rangle \ : \ C \text{ is an independent set of size } k \text{ in } G\}$$

Is INDSET-verify in P? Yes!

1. Check that $|C| = k$.
2. For each $u, v \in C$:
   - Check $\{u, v\} \notin E(G)$.

$$\begin{aligned} \text{INDSET} &= \{\langle G, k\rangle \ : \ G \text{ has an independent of size } k\} \\ &= \{\langle G, k\rangle \ : \ \exists C \text{ such that } \langle G, k, C\rangle \in \text{INDSET-verify}\} \end{aligned}$$

INDSET-verify and INDSET

$$\text{INDSET-verify} = \{\langle G, k, C\rangle \ : \ C \text{ is an independent set of size } k \text{ in } G\}$$

Is INDSET-verify in P? Yes!

1. Check that $|C| = k$.
2. For each $u, v \in C$:
   - Check $\{u, v\} \notin E(G)$.

$$\begin{aligned} \text{INDSET} &= \{\langle G, k\rangle \ : \ G \text{ has an independent of size } k\} \\ &= \{\langle G, k\rangle \ : \ \exists C \text{ such that } \langle G, k, C\rangle \in \text{INDSET-verify}\} \end{aligned}$$

Is INDSET in P?

INDSET-verify and INDSET

$$\text{INDSET-verify} = \{\langle G, k, C\rangle \ : \ C \text{ is an independent set of size } k \text{ in } G\}$$

Is INDSET-verify in P? Yes!

1. Check that $|C| = k$.
2. For each $u, v \in C$:
   - Check $\{u, v\} \notin E(G)$.

$$\begin{aligned} \text{INDSET} &= \{\langle G, k\rangle \ : \ G \text{ has an independent of size } k\} \\ &= \{\langle G, k\rangle \ : \ \exists C \text{ such that } \langle G, k, C\rangle \in \text{INDSET-verify}\} \end{aligned}$$

Is INDSET in P?

Decider for INDSET:

1. For each subset $C \subseteq V(G)$:
   - If $\langle G, k, C\rangle \in$ INDSET-verify, ACCEPT $\langle G, k\rangle$.
2. REJECT $\langle G, k\rangle$.

Verifiers and the class NP

Recall: A **decider** for language L is a TM M such that for each $x \in \Sigma^*$

- If $x \in L$, then M accepts x.
- If $x \notin L$, then M rejects x.

Verifiers and the class NP

Recall: A **decider** for language L is a TM M such that for each $x \in \Sigma^*$

- If $x \in L$, then M accepts x.
- If $x \notin L$, then M rejects x.

Definition:
A **verifier** for language L is a TM M such that for each $x \in \Sigma^*$

- If $x \in L$, then there exists C such that M accepts $\langle x, C \rangle$.
- If $x \notin L$, then for every C, M rejects $\langle x, C \rangle$.

(C is called a *certificate* or *witness*)

Verifiers and the class NP

Recall: A **decider** for language L is a TM M such that for each $x \in \Sigma^*$

- If $x \in L$, then M accepts x.
- If $x \notin L$, then M rejects x.

Definition:
A **verifier** for language L is a TM M such that for each $x \in \Sigma^*$

- If $x \in L$, then there exists C such that M accepts $\langle x, C \rangle$.
- If $x \notin L$, then for every C, M rejects $\langle x, C \rangle$.

(C is called a *certificate* or *witness*)

A verifier is a **polynomial time** verifier if its running time on any $\langle x, C \rangle$ is **polynomial in** $|\mathbf{x}|$. (Thus $|C|$ is polynomial in $|x|$)

Verifiers and the class NP

Recall: A **decider** for language L is a TM M such that for each $x \in \Sigma^*$

- If $x \in L$, then M accepts x.
- If $x \notin L$, then M rejects x.

Definition:
A **verifier** for language L is a TM M such that for each $x \in \Sigma^*$

- If $x \in L$, then there exists C such that M accepts $\langle x, C \rangle$.
- If $x \notin L$, then for every C, M rejects $\langle x, C \rangle$.

(C is called a *certificate* or *witness*)

A verifier is a **polynomial time** verifier if its running time on any $\langle x, C \rangle$ is **polynomial in** $|\mathbf{x}|$. (Thus $|C|$ is polynomial in $|x|$)

Definition: NP is the class of languages that have poly-time verifiers.

Why is it called **NP**?

Detour: Non-deterministic Turing Machines

Recall: In a Turing machine, $\delta\colon (Q \times \Gamma) \longrightarrow Q \times \Gamma \times \{L, R\}$.

Detour: Non-deterministic Turing Machines

Recall: In a Turing machine, $\delta\colon (Q \times \Gamma) \longrightarrow Q \times \Gamma \times \{L, R\}$.

In a **Nondeterministic Turing Machine (NTM)**,

$$\delta : (Q \times \Gamma) \longrightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

(several possible transitions for a given state and tape symbol)

Detour: Non-deterministic Turing Machines

Recall: In a Turing machine, $\delta\colon (Q \times \Gamma) \longrightarrow Q \times \Gamma \times \{L, R\}$.

In a **Nondeterministic Turing Machine (NTM)**,

$$\delta : (Q \times \Gamma) \longrightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

(several possible transitions for a given state and tape symbol)

Definition: A **nondeterministic decider** for language L is an NTM N such that for each $x \in \Sigma^*$, **every computation of N on x halts**, and moreover,

- If $x \in L$, then some computation of N on x accepts.
- If $x \notin L$, then every computation of N on x rejects.

Detour: Non-deterministic Turing Machines

Recall: In a Turing machine, $\delta\colon (Q \times \Gamma) \longrightarrow Q \times \Gamma \times \{L, R\}$.

In a **Nondeterministic Turing Machine (NTM)**,

$$\delta : (Q \times \Gamma) \longrightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

(several possible transitions for a given state and tape symbol)

Definition: A **nondeterministic decider** for language L is an NTM N such that for each $x \in \Sigma^*$, **every computation of N on x halts**, and moreover,

- If $x \in L$, then some computation of N on x accepts.
- If $x \notin L$, then every computation of N on x rejects.

An NTM is a **polynomial time** NTM if the running time of its longest computation on x is **polynomial in $|\mathbf{x}|$**.

Nondeterministic deciders $\iff$ Verifiers

Theorem: For any language $L \subseteq \Sigma^*$,

L has a nondeterministic poly-time decider $\iff$ L has a poly-time verifier.

Nondeterministic deciders $\iff$ Verifiers

Theorem: For any language $L \subseteq \Sigma^*$,

L has a nondeterministic poly-time decider $\iff$ L has a poly-time verifier.

Proof Sketch ($\Longleftarrow$):

Let M be the verifier. NTM N on input x does the following:

Nondeterministic deciders $\iff$ Verifiers

Theorem: For any language $L \subseteq \Sigma^*$,

L has a nondeterministic poly-time decider $\iff$ L has a poly-time verifier.

Proof Sketch ($\Longleftarrow$):

Let M be the verifier. NTM N on input x does the following:

1. Write a certificate C nondeterministically.
2. Run M on $\langle x, C \rangle$.

Nondeterministic deciders $\iff$ Verifiers

Theorem: For any language $L \subseteq \Sigma^*$,

L has a nondeterministic poly-time decider $\iff$ L has a poly-time verifier.

Proof Sketch ($\Longleftarrow$):

Let M be the verifier. NTM N on input x does the following:

1. Write a certificate C nondeterministically.
2. Run M on $\langle x, C \rangle$.

Proof Sketch ($\Longrightarrow$):

Let N be the nondeterministic decider. Verifier M on $\langle x, C \rangle$ computes:

Nondeterministic deciders $\iff$ Verifiers

Theorem: For any language $L \subseteq \Sigma^*$,

L has a nondeterministic poly-time decider $\iff$ L has a poly-time verifier.

Proof Sketch ($\Longleftarrow$):

Let M be the verifier. NTM N on input x does the following:

1. Write a certificate C nondeterministically.
2. Run M on $\langle x, C \rangle$.

Proof Sketch ($\Longrightarrow$):

Let N be the nondeterministic decider. Verifier M on $\langle x, C \rangle$ computes:

- Simulate N on x, choosing transitions given by C.

Nondeterministic deciders $\iff$ Verifiers

Theorem: For any language $L \subseteq \Sigma^*$,

L has a nondeterministic poly-time decider $\iff$ L has a poly-time verifier.

Proof Sketch ($\Longleftarrow$):

Let M be the verifier. NTM N on input x does the following:

1. Write a certificate C nondeterministically.
2. Run M on $\langle x, C \rangle$.

Proof Sketch ($\Longrightarrow$):

Let N be the nondeterministic decider. Verifier M on $\langle x, C \rangle$ computes:

- Simulate N on x, choosing transitions given by C.

M accepts $\langle x, C \rangle$ iff $x \in L$ and C is an accepting path of N on x.

Non-deterministic Polynomial-time

Theorem: For any language $L \subseteq \Sigma^*$,

L has a nondeterministic poly-time decider $\iff$ L has a poly-time verifier.

Non-deterministic Polynomial-time

Theorem: For any language $L \subseteq \Sigma^*$,

L has a nondeterministic poly-time decider $\iff$ L has a poly-time verifier.

Definition: **NP** is the class of languages which have poly-time nondeterministic deciders, or equivalently, have poly-time verifiers.

Non-deterministic Polynomial-time

Theorem: For any language $L \subseteq \Sigma^*$,

L has a nondeterministic poly-time decider $\iff$ L has a poly-time verifier.

Definition: **NP** is the class of languages which have poly-time nondeterministic deciders, or equivalently, have poly-time verifiers.

Definition:

$$\mathrm{NTIME}(t(n)) = \{L \ : \ L \text{ has a nondeterministic } O(t(n)) \text{ time decider}\}$$

Non-deterministic Polynomial-time

Theorem: For any language $L \subseteq \Sigma^*$,

L has a nondeterministic poly-time decider $\iff$ L has a poly-time verifier.

Definition: **NP** is the class of languages which have poly-time nondeterministic deciders, or equivalently, have poly-time verifiers.

Definition:

$$\mathrm{NTIME}(t(n)) = \{L \ : \ L \text{ has a nondeterministic } O(t(n)) \text{ time decider}\}$$

Then

$$\mathrm{NP} = \bigcup_{k=1}^{\infty} \mathrm{NTIME}(n^k)\,.$$

Non-deterministic Polynomial-time

Theorem: For any language $L \subseteq \Sigma^*$,

L has a nondeterministic poly-time decider $\iff$ L has a poly-time verifier.

Definition: **NP** is the class of languages which have poly-time nondeterministic deciders, or equivalently, have poly-time verifiers.

Definition:

$$\mathrm{NTIME}(t(n)) = \{L \ : \ L \text{ has a nondeterministic } O(t(n)) \text{ time decider}\}$$

Then

$$\mathrm{NP} = \bigcup_{k=1}^{\infty} \mathrm{NTIME}(n^k)\,.$$

$\therefore$ SAT, GI, INDSET are all in NP.

P and NP

Is $P \subseteq NP$? Yes (obviously)

P and NP

Is $\mathsf{P} \subseteq \mathsf{NP}$? Yes (obviously)

Is $\mathsf{P} = \mathsf{NP}$? Nobody knows ...

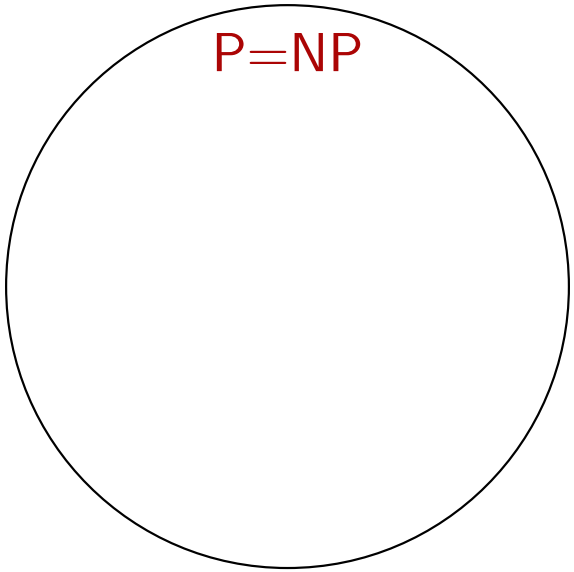
Find the answer and win USD 1,000,000!

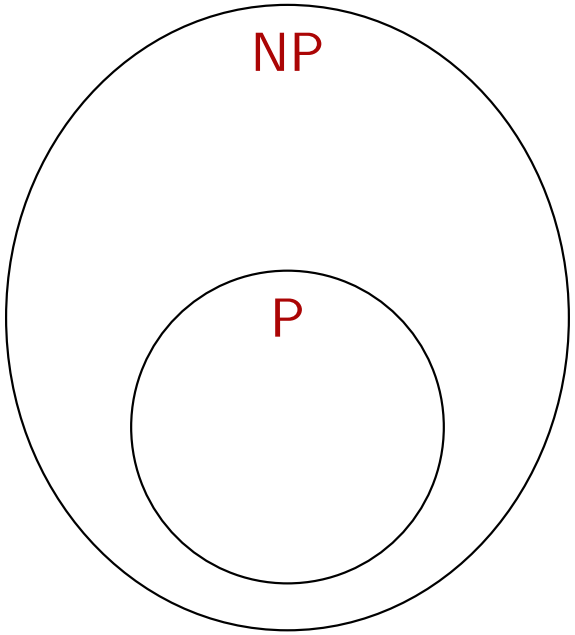
P and NP

Is P $\subseteq$ NP? Yes (obviously)

Is P $=$ NP? Nobody knows ...

Find the answer and win USD 1,000,000!



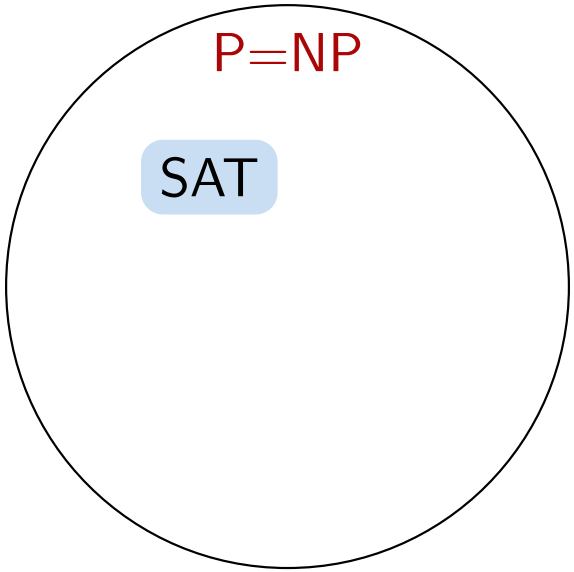


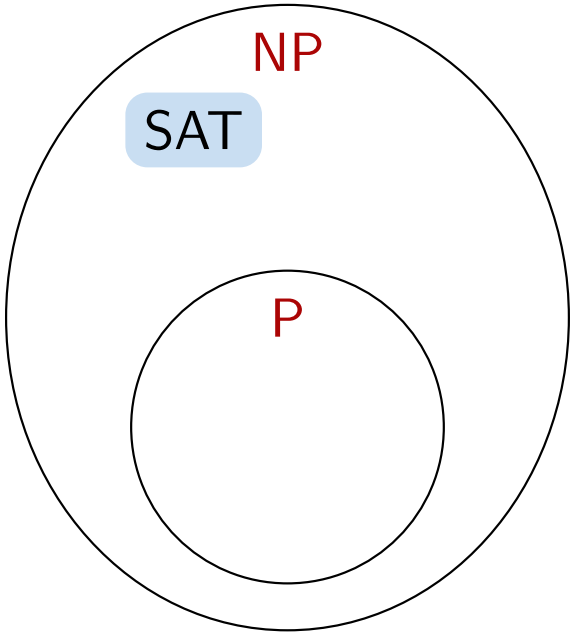
P and NP

Is P $\subseteq$ NP? Yes (obviously)

Is P $=$ NP? Nobody knows ...

Find the answer and win USD 1,000,000!



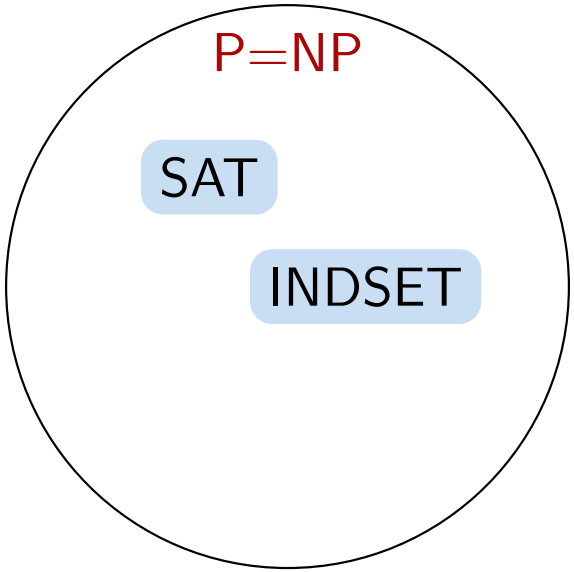


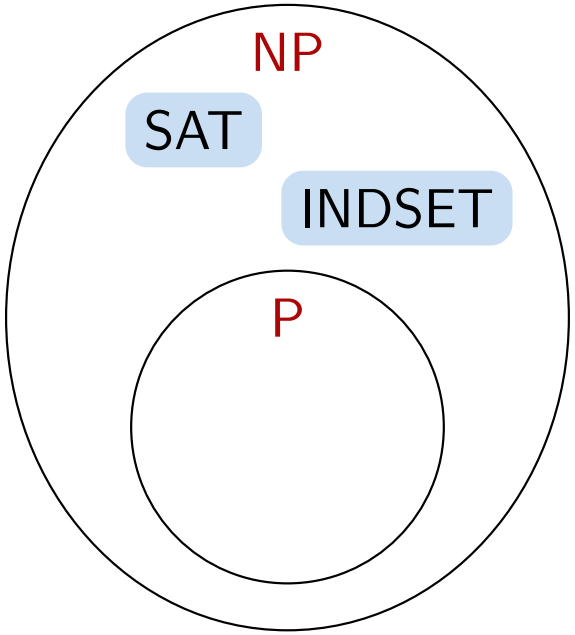
P and NP

Is P $\subseteq$ NP? Yes (obviously)

Is P $=$ NP? Nobody knows ...

Find the answer and win USD 1,000,000!



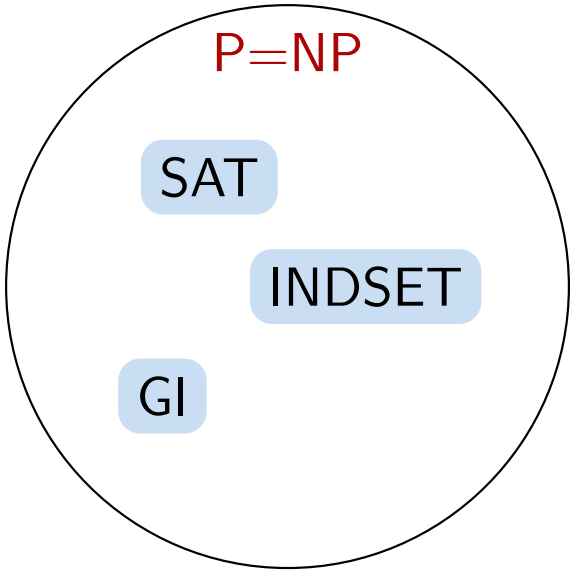


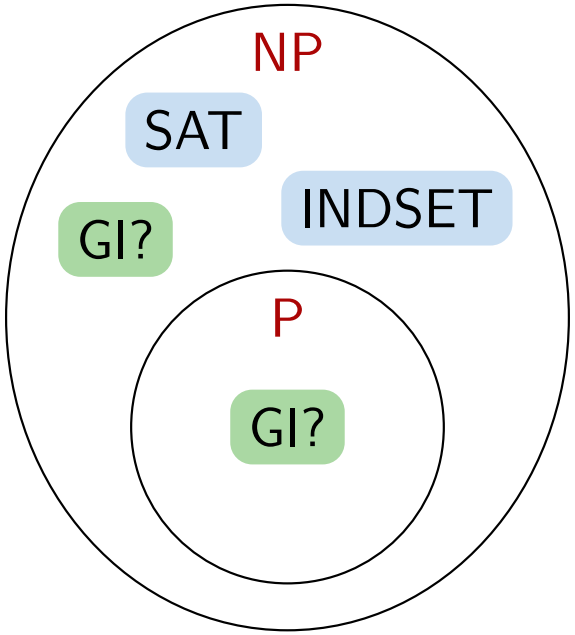
P and NP

Is P $\subseteq$ NP? Yes (obviously)

Is P $=$ NP? Nobody knows ...

Find the answer and win USD 1,000,000!



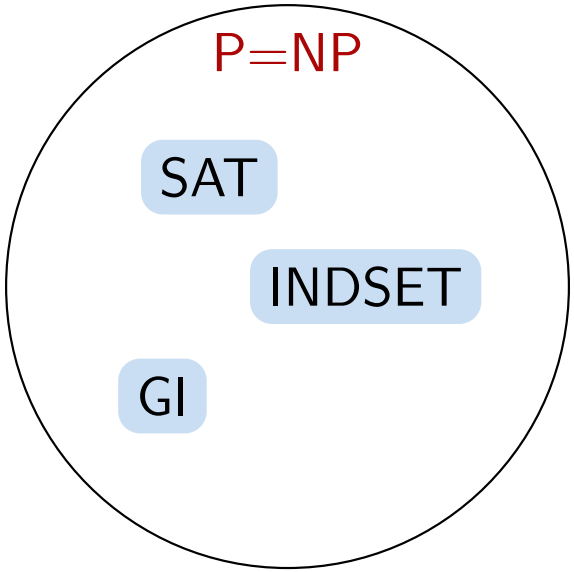


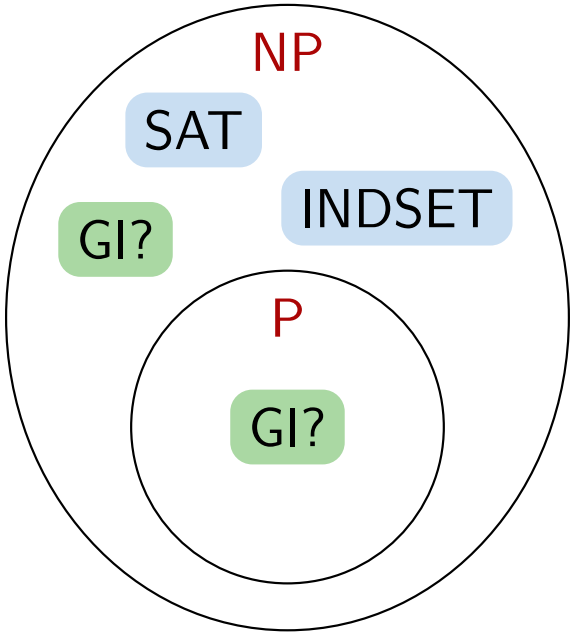
P and NP

Is $P \subseteq NP$? Yes (obviously)

Is $P = NP$? Nobody knows ...

Find the answer and win USD 1,000,000!





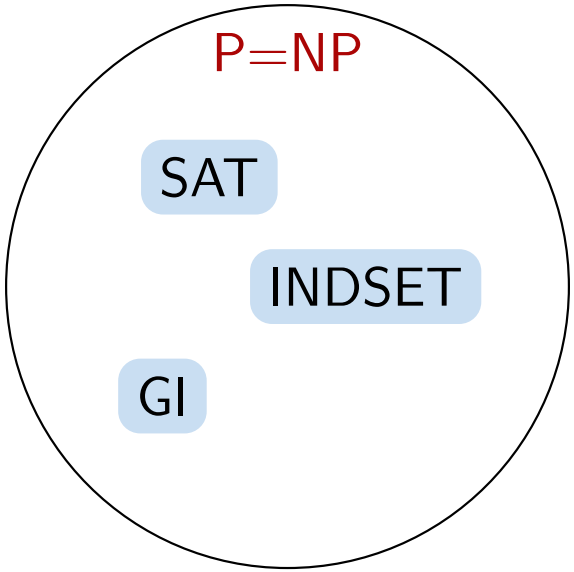
Cook-Levin Theorem (informal): $SAT \in P$ iff $P = NP$.

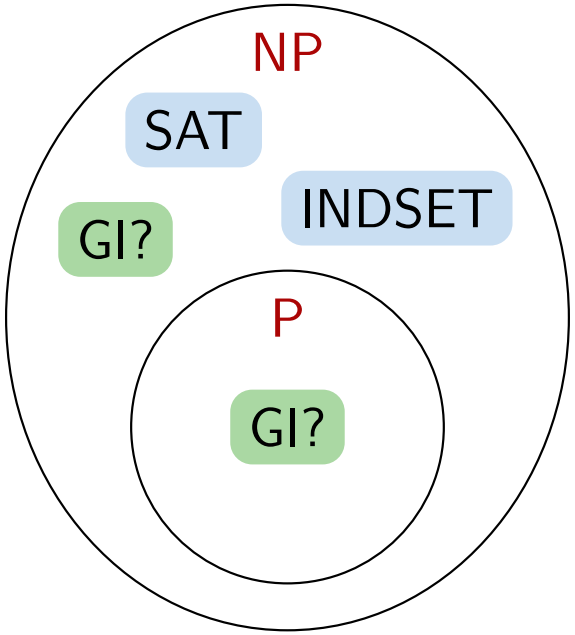
P and NP

Is $P \subseteq NP$? Yes (obviously)

Is $P = NP$? Nobody knows ...

Find the answer and win USD 1,000,000!





Cook-Levin Theorem (informal): $SAT \in P$ iff $P = NP$.

(Also $INDSET \in P$ iff $P = NP$.)